Contents

Olist — Module 3 Lab Guide

Scenario: Brazilian E-commerce. Predict late deliveries. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **Olist**. Olist is the biggest online marketplace in Brazil. Like Amazon, but Brazilian.

Every month: - **100,000 customers** buy something on Olist. - Sellers from all over Brazil ship the orders. - Each customer is promised a delivery date.

The problem

About **7% of orders arrive AFTER the promised date.** That is **7,000 angry customers every month.**

Angry customers: - Write 1-star reviews. - Tell their friends to use a competitor. - Never buy from Olist again.

The CEO is worried. She calls your team into a meeting on Monday morning.

Your manager's request

Your manager, **Beatriz** (Head of Logistics), tells you:

> "I do not need you to make the trucks faster. I cannot change geography.
>
> I need a different tool. When a new order is placed, I want a number: **'this order has a 70% chance of being late.'**
>
> Then we will do TWO things: 1. Send the customer a polite email: 'Sorry, your order may be late. Here is a $5 voucher.' 2. Call the seller and ask them to ship faster.
>
> If we catch even half the late orders early, we save thousands of customers."

Your team's job for the next 2 weeks (Module 3)

Beatriz cannot do this alone. Her data is **9 messy CSV files** sitting in different systems.

Your job in Module 3: > **Turn 9 messy CSV files into ONE clean file. The clean file will be used to train the model in Module 4.**

The clean file is called `olist_clean.parquet`. It must have **21 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the prediction model. Beatriz finally gets her "late-risk score".
Module 5	Find groups of customers (VIPs vs at-risk vs new). For marketing.
Module 7	Read the review comments (in Portuguese). Find common complaints.

You use the **same Olist dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ___ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

> Goal: **"What does this data look like?"**

2. Explanatory charts (for BEATRIZ)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

> Goal: **"Beatriz, look at this. This is the problem."**

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	`plt.hist()` or `sns.histplot()`
Bar chart	Count of each category	`df['col'].value_counts().plot.bar()`
Box plot	See outliers in a numeric column	`sns.boxplot()`
Scatter plot	See the relationship between 2 numbers	`plt.scatter()` or `sns.scatterplot()`
Heatmap	See correlation between many columns	`sns.heatmap(df.corr())`
Line plot	See change over time	`plt.plot()`

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to https://colab.research.google.com
2. Sign in with your Google account.
3. Click **"New notebook"** (top left).
4. Name it `olist_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click "Allow". After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/olist_lab', exist_ok=True)
%cd /content/drive/MyDrive/olist_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

Option A — Direct from Kaggle (recommended):

1. Make a free Kaggle account.
2. Go to kaggle.com/settings/account, scroll to "API", click "Create New Token". Save the `kaggle.json` file.
3. In Colab:

```
from google.colab import files
files.upload()    # pick kaggle.json
```

4. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d olistbr/brazilian-ecommerce
!unzip -q brazilian-ecommerce.zip -d data
!ls data/
```

You should see **9 CSV files** inside `data/`.

Option B — Upload by hand:

1. Download the zip from kaggle.com/datasets/olistbr/brazilian-ecommerce on your laptop.
2. Unzip it.
3. In Colab's file panel (left sidebar), upload all 9 CSV files.

Step 5 — Test it

```
import pandas as pd
orders = pd.read_csv('data/olist_orders_dataset.csv')
print(orders.shape)
```

Should print `(99441, 8)`. You are ready.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate `.parquet` files to Drive	Files in `/content/` disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime > Restart runtime.

Class 1 — Data Cleaning

> **Scenario reminder:** Beatriz drops 9 messy CSV files on your desk. Some date columns are stored as text. Some rows have missing values. Your job today: clean the most important file (`orders`).

Your goal

Make the 9 CSV files USABLE. Fix date columns. Find missing values. Decide what to keep.

Inputs

- The 9 CSV files in `data/`

Outputs

- `orders_step1.parquet` saved in your Drive folder
- 3+ exploratory charts in your notebook
- 1 explanatory chart for Beatriz
- Notes in markdown about every decision you made

Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data.

Exploratory chart 1 — How many orders per status?

- **Question:** "Of 99,441 orders, how many are `delivered`, how many `canceled`, etc.?"
- **HINTS:**
  - Use `orders['order_status'].value_counts()`.
  - Then `.plot.bar()` on the result.
- **What you learn:** Most orders are `delivered`. We will keep only those.

Exploratory chart 2 — When were orders placed (by month)?

- **Question:** "When in the year do most orders happen? Is there a holiday spike?"
- **HINTS:**
  - First convert `order_purchase_timestamp` to datetime (you do this in Phase B Step 3 anyway, do it here too).
  - Use `.dt.to_period('M')` to group by month.
  - Plot the counts.
- **What you learn:** Does Olist have a December rush? A weekly pattern?

Exploratory chart 3 — Missing values per column

- **Question:** "Which columns have the most missing data?"
- **HINTS:**
  - Use `orders.isna().sum().sort_values(ascending=False)`.
  - Plot it as a bar chart.
- **What you learn:** Which columns will be hard to use.

Phase B — Clean the orders table (45 minutes)

Step 1 — Load all 9 files

- **WHAT:** Load each of the 9 CSV files into its own DataFrame.
- **HINTS:**
  - Use `pd.read_csv('data/<filename>')`.
  - Give each DataFrame a short name: `orders`, `customers`, `items`, `payments`, `reviews`, `products`, `sellers`, `geo`, `trans`.
- **EXPECTED:** 9 DataFrames in memory.

Step 2 — Look at each DataFrame

- **WHAT:** For each, check `.shape`, `.info()`, and `.head()`.
- **HINTS:**
  - Loop over your 9 DataFrames? Or just call them one by one.
  - Look at the `Dtype` column in `.info()` output. **Are dates stored as `object` (text)?**
- **EXPECTED:**

DataFrame	Approx rows
orders	99,441
customers	99,441
items	112,650
payments	103,886
reviews	99,224
geo	1,000,163

Step 3 — Fix the date columns

- **WHAT:** In `orders`, **5 date columns** are stored as text. Convert them to real datetime.
- **The 5 columns:**
  - `order_purchase_timestamp`
  - `order_approved_at`
  - `order_delivered_carrier_date`
  - `order_delivered_customer_date`
  - `order_estimated_delivery_date`
- **HINTS:**
  - The function is `pd.to_datetime()`.
  - Add the argument `errors='coerce'`. If a cell is bad, it becomes NaT (Not a Time = missing). The code does not crash.
  - Use a `for` loop over the 5 column names. Do NOT write 5 separate lines.
- **WHY:** If dates are strings, you cannot subtract them. "How many days from purchase to delivery?" is impossible.
- **EXPECTED:** After your code, `orders.dtypes` shows `datetime64[ns]` for those 5 columns.

Step 4 — Find missing values

- **WHAT:** Count missing values per column.

- **HINTS:** `.isna()` returns True/False per cell. `.sum()` counts the Trues per column.
- **EXPECTED:** Something like:

```
order_approved_at               160
order_delivered_carrier_date   1783
order_delivered_customer_date  2965
```

Step 5 — Filter to delivered orders

- **WHAT:** Keep only rows where `order_status == 'delivered'`.
- **HINTS:**
  - Boolean mask: `orders[orders['order_status'] == 'delivered']`.
  - Add `.copy()` at the end. This avoids a warning later.
- **WHY:** Only delivered orders have a real delivery date. We cannot compute "late?" for the others.
- **EXPECTED:** About **96,478 rows** left.

Step 6 — Drop rows missing the delivery date

- **WHAT:** Even after Step 5, a few rows still have no `order_delivered_customer_date`. Drop them.
- **HINTS:**
  - Use `.dropna(subset=['order_delivered_customer_date'])`.
- **WHY:** We cannot compute `is_late` without the actual delivery date.
- **EXPECTED:** About **96,470 rows** left.

Step 7 — Write down what you did

In a markdown cell, write: - Starting rows: ~99,441 - After status filter: ~96,478 - After dropping missing dates: ~96,470 - WHY you removed each group.

Step 8 — Save to Drive

- **WHAT:** Save the cleaned `orders` DataFrame as parquet, inside your Drive folder.
- **HINTS:** `df.to_parquet('/content/drive/MyDrive/olist_lab/orders_step1.parquet')`.

Phase C — Make ONE chart for Beatriz (15 minutes)

She does not have time to read your code. She wants ONE picture.

Beatriz's chart — "How clean was the data?"

Make a bar chart showing the row count after each cleaning step:

Step	Rows
Raw orders	99,441
After "delivered" filter	96,478
After drop-missing-date	96,470

- **HINTS:**
  - Use `plt.bar()` with 3 bars.
  - Add the title: `"Data cleaning impact — 99.4% of orders survived"`.
  - X-label: stages of cleaning.
  - Y-label: row count.
  - Put the number on top of each bar.

- **Takeaway for Beatriz:** “We threw out less than 1% of the data. Almost everything is usable.”

Common mistakes in Class 1

Mistake	What goes wrong
Forget `errors='coerce'` on `to_datetime`	Code crashes on one bad date.
Forget `.copy()` after a filter	Pandas warns “SettingWithCopyWarning”.
Delete rows without writing WHY	Module 4 students will not understand.
Save to `/content/` instead of Drive	File disappears when Colab disconnects.

Self-check before Class 2

- ☐ All 9 DataFrames loaded.
- ☐ The 5 date columns in `orders` have dtype `datetime64`.
- ☐ You filtered to delivered orders.
- ☐ At least 3 exploratory charts in your notebook.
- ☐ 1 polished chart for Beatriz.
- ☐ You saved `orders_step1.parquet` to your Drive folder.
- ☐ You wrote down (in markdown) what you did and WHY.

Class 2 — Encoding and Scaling

> **Scenario reminder:** Beatriz looks at your cleaned data. She is happy. But she says: “the model is a math model. It does not understand the word ‘credit_card’. Turn the words into numbers.”

Your goal

Turn TEXT columns into numbers. Make all numeric columns about the same size.

Inputs

- `orders_step1.parquet` from Class 1
- The other 8 raw CSVs (still needed for joins)

Outputs

- `orders_step2.parquet` in Drive
- 3+ exploratory charts
- 1 explanatory chart for Beatriz

Phase A — Explore the data first

Exploratory chart 1 — Distribution of `total_price`

- **Question:** “Most orders are small ($50). A few are huge ($5000+). Is the distribution skewed?”
- **HINTS:**
  - Use a histogram (`plt.hist()`).
  - Use `bins=50` to see the shape.
- **What you learn:** The price column has a “long tail”. This is why we will use log-transform.

Exploratory chart 2 — Distribution of `total_price` AFTER `np.log1p`

- **Question:** "Does log-transform make the shape more normal?"
- **HINTS:**
  - Make a NEW column: `log_price = np.log1p(df['total_price'])`.
  - Histogram it.
  - Compare to chart 1.
- **What you learn:** Log makes the long tail manageable for the model.

Exploratory chart 3 — `payment_type` counts

- **Question:** "Which payment methods are popular in Brazil?"
- **HINTS:** `value_counts().plot.bar()`.
- **What you learn:** Are some categories super rare? (Should you merge them?)

Exploratory chart 4 — `customer_state` counts

- **Question:** "How many states do customers come from?"
- **HINTS:** `value_counts().plot.bar()`. Or `sns.countplot()`.
- **What you learn:** 27 states. Sao Paulo dominates. This affects encoding choice.

Phase B — Encode and scale

Step 1 — Join the most important tables

- **WHAT:** Merge `orders` + `customers` + `items` + `payments` into one DataFrame `df`.
- **HINTS:**
  - Use `df = orders.merge(customers, on='customer_id', how='left')`.
  - Repeat for items (`on='order_id'`) and payments (`on='order_id'`).
- **WHY `how='left'`?** Keep all orders even if some have missing items.
- **EXPECTED:** ~112,650 rows (one per ORDER ITEM, not per order — `items` expands the table).

Step 2 — Aggregate to one row per order

- **WHAT:** Group by `order_id`. Sum the numeric columns. Take the first value for text columns.
- **HINTS:**
  - Use `df.groupby('order_id').agg(...)`.
  - Sum: `total_price`, `total_freight`, `num_items` (count of items).
  - First: `payment_type`, `customer_state`, the date columns.
- **EXPECTED:** ~96,470 rows × ~12 columns.

Step 3 — One-hot encode `payment_type`

- **WHAT:** Turn the 5 categories into 5 new 0/1 columns.
- **HINTS:**
  - Use `pd.get_dummies(df, columns=['payment_type'], prefix='payment')`.
- **EXPECTED:** 5 new columns: `payment_credit_card`, `payment_boleto`, etc.

Step 4 — Decide what to do with `customer_state`

- **WHAT:** 27 states. Too many for one-hot? Maybe.
- **TWO OPTIONS:**
  - **A — One-hot:** Add 27 new 0/1 columns.
  - **B — Target encoding:** Replace each state with the AVG `is_late` rate for that state (compute from train only).

- **YOUR CHOICE:** Pick one. Write in your notebook WHY.

Step 5 — Log-transform `total_price` and `total_freight`

- **WHAT:** Both columns have very long tails. Apply `np.log1p()`.
- **HINTS:** `df['log_price'] = np.log1p(df['total_price'])`.
- **WHY `log1p` and not `log`?** `log(0)` is `-inf`. `log1p(x) = log(x + 1)` is safe for zeros.

Step 6 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
  - `from sklearn.preprocessing import StandardScaler`.
  - `scaler = StandardScaler()`.
  - `scaler.fit_transform(df[numeric_cols])`.
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

Step 7 — Save

- **WHAT:** Save to Drive. `df.to_parquet('/content/drive/MyDrive/olist_lab/orders_step2.parquet')`.
- **Save the UNSCALED version.** Scaling will happen inside the Pipeline later.

Phase C — Make ONE chart for Beatriz

Beatriz's chart — "Where do our customers live?"

A bar chart of the top 10 states by order count.

- **HINTS:**
  - `df['customer_state'].value_counts().head(10).plot.bar()`.
- **Title:** "Top 10 states by order volume — Sao Paulo dominates (40%)".
- **X-label:** State.
- **Y-label:** Number of orders.
- **Takeaway:** "Most customers are in 5 states. Your trucks should focus there."

Common mistakes in Class 2

Mistake	What goes wrong
Scale BEFORE train/test split	**Leakage.** Scaler sees test data.
One-hot encode every text column	Some columns have 1000+ values. Table explodes.
Forget `np.log1p` and use `np.log` on zeros	`np.log(0) = -inf`. Crash.
Save the scaled version	Class 5 will scale again inside the pipeline. Double scaling = wrong numbers.

Self-check before Class 3

- ☐ One row per order.
- ☐ `payment_type` encoded.
- ☐ You decided what to do with `customer_state`.
- ☐ `log_price` and `log_freight` exist.
- ☐ At least 3 exploratory charts.

☐ 1 chart for Beatriz.
☐ `orders_step2.parquet` saved to Drive.

Class 3 — Feature Engineering

> **Scenario reminder:** Beatriz says: "The columns in the raw data are not enough. The TRULY useful columns are not there. We must MAKE them. For example, the distance between buyer and seller — biggest predictor of lateness."

Your goal

Make NEW columns from the existing ones. These will help the model predict lateness.

Inputs

- `orders_step2.parquet`
- `geolocation` (zip code -> lat/lon)
- `sellers`

Outputs

- `orders_step3.parquet` in Drive
- 3+ exploratory charts
- 1 explanatory chart for Beatriz

Phase A — Explore the data first

Exploratory chart 1 — `is_late` distribution

- First make the `is_late` column (see Step 1 below).
- **Question:** "What % of orders are late?"
- **HINTS:** `df['is_late'].value_counts(normalize=True).plot.bar()`.
- **What you learn:** Confirm it is around 5-10%.

Exploratory chart 2 — `is_late` by purchase day of week

- **Question:** "Are weekend orders more likely to be late?"
- **HINTS:**
  - Make `purchase_dayofweek` (see Step 2 below).
  - Use `df.groupby('purchase_dayofweek')['is_late'].mean()`.
  - Plot as a bar chart.
- **What you learn:** Patterns by day of week.

Exploratory chart 3 — Distance histogram

- After you compute `distance_km` (Step 4), look at its distribution.
- **HINTS:** Histogram with `bins=50`.
- **What you learn:** Most orders are < 500 km. A few are 3000+ km (Brazil is huge).

Exploratory chart 4 — Distance vs late rate

- **Question:** "Are far orders more likely to be late?"
- **HINTS:**
  - Group `distance_km` into 10 bins (`pd.cut()`).
  - Compute `is_late` mean per bin.
  - Bar chart.
- **What you learn:** This is the most important feature.

Phase B — Engineer the features

Step 1 — Create the target column `is_late`

- **WHAT:** `is_late = 1` if delivered AFTER the estimated date, else 0.
- **HINTS:**
  - Compare two columns: `delivered_customer_date > estimated_delivery_date`.
  - The result is True/False. Convert to int with `.astype(int)`.
- **EXPECTED:** About 7% of rows have `is_late = 1`. Confirm with `df['is_late'].mean()`.

Step 2 — Date-derived features

Make these new columns from `order_purchase_timestamp`:

New column	What it is
`purchase_year`	The year
`purchase_month`	The month (1–12)
`purchase_dayofweek`	0=Monday, 6=Sunday
`purchase_hour`	0–23
`is_weekend`	1 if dayofweek >= 5, else 0

- **HINTS:**
  - Use the `.dt` accessor on a datetime column.
  - `df['col'].dt.year`, `.dt.month`, `.dt.dayofweek`, `.dt.hour`.
  - For `is_weekend`, write a comparison and convert to int.
- **WHY:** A model can learn "Friday-night orders are riskier" only if YOU give it the `purchase_dayofweek` column.

Step 3 — Date-difference features

New column	What it is
`delivery_days`	Actual delivery time, in days
`estimate_days`	Promised delivery time, in days

- **HINTS:**
  - Subtract two datetime columns. Result is a Timedelta.
  - Use `.dt.days` on the Timedelta to get a number.
- **WARNING:**
  - `estimate_days` is OK to use as a feature. (We know the estimate at order time.)
  - `delivery_days` is NOT OK. Knowing the actual delivery time = knowing the answer. This is **leakage.** Use for analysis only.

Step 4 — Compute distance with the Haversine formula

This is the **biggest** feature.

Sub-step 4a — Build `zip -> (lat, lon)` map from geo: - **HINTS:** - `geo.groupby('geolocation_zip_code_prefix').ag` `lon=...)`. - Use `'mean'` for both lat and lon (sometimes one zip has many points).

Sub-step 4b — Join the map to customers and sellers. - **HINTS:** - Merge `zip_to_latlon` with `customers` on `customer_zip_code_prefix == geolocation_zip_code_prefix`. - Repeat for sellers. - Then join customer_lat/lon and seller_lat/lon to your main `df`.

Sub-step 4c — Compute Haversine distance.

You need to write the formula. Skeleton:

```
import numpy as np
def haversine(lat1, lon1, lat2, lon2):
    R = 6371                    # Earth radius in km
    phi1 = np.radians(___)      # YOU fill in
    phi2 = np.radians(___)
    dphi = np.radians(___ - ___)
    dlam = np.radians(___ - ___)
    a = np.sin(dphi/2)**2 + np.cos(phi1)*np.cos(phi2) * np.sin(dlam/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))
```

Then call it: `df['distance_km'] = haversine(...)` with the 4 columns.

- **WHY Haversine?** The Earth is a sphere. Straight-line distance on a flat map is wrong. Haversine gives the real distance.
- **EXPECTED:** `distance_km` between 0 and ~5000.

Step 5 — Domain features (pick at least 2)

New column	What it is
`freight_per_item`	`total_freight / num_items`
`price_per_item`	`total_price / num_items`
`freight_ratio`	`total_freight / (total_price + 1)`
`seller_avg_late_rate`	For each seller, the % of past orders late

- **HINTS:**
  - Simple arithmetic for the first 3.
  - For `seller_avg_late_rate`: groupby `seller_id`, compute mean of `is_late`. WARNING: use train data only.

Step 6 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/olist_lab/orders_step3.parquet')`.

Phase C — Make ONE chart for Beatriz

Beatriz's chart — "Distance kills delivery times"

A bar chart showing: bin distance into 5 buckets, show the % late in each bucket.

- **HINTS:**
  - `pd.cut(df['distance_km'], bins=[0, 100, 500, 1000, 2000, 5000])`.
  - GroupBy that, take the mean of `is_late`.
  - Multiply by 100 to get %.

  - Bar chart.
- **Title:** “Late delivery rate by distance — orders over 1000 km are 4x more likely to be late.”
- **Takeaway for Beatriz:** “If we promise faster delivery on long routes, we lose money. Maybe extend the estimate for distant orders.”

Common mistakes in Class 3

Mistake	What goes wrong
Use `delivery_days` as a feature	Leakage. The model will look 100% accurate but fail in production.
Forget `.dt.days` and get a Timedelta	Model crashes on Timedelta dtype.
Use Euclidean distance instead of Haversine	Wrong distances. Brazil is huge and curved.
Compute seller late rate from ALL data	Leakage. Only from train data.

Self-check before Class 4

- ☐ `is_late` exists. Mean ~7%.
- ☐ 5 date-derived features exist.
- ☐ `distance_km` looks reasonable (0–5000).
- ☐ At least 2 domain features.
- ☐ 3+ exploratory charts.
- ☐ 1 explanatory chart.
- ☐ `orders_step3.parquet` saved to Drive.

Class 4 — Feature Selection

> **Scenario reminder:** Now you have ~25 columns. Beatriz says: “Too many. Some are duplicates. Some are useless. I want 10–15 GOOD columns. Pick them.”

Your goal

Pick the best 10–15 columns. Drop the rest. Justify every choice.

Inputs

- `orders_step3.parquet`

Outputs

- `orders_step4.parquet` in Drive (only the selected columns + `is_late`)
- 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
- A short markdown report explaining your choices

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”

- **HINTS:**
  - Compute `df[numeric_cols].corr()`.
  - Plot with `sns.heatmap()`.
  - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** Pairs of columns with |corr| > 0.9 are redundant.

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
  - Sort the values, then bar chart.
- **What you learn:** Which columns predict `is_late` strongest.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
- **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
- **What you learn:** A second opinion on feature importance.

Phase B — Select features

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
  - `from sklearn.model_selection import train_test_split`.
  - `X = df.drop('is_late', axis=1); y = df['is_late']`.
  - Pass: `test_size=0.2`, `random_state=42`, `stratify=y`.
- **WHY:** All next steps use TRAIN ONLY. Otherwise leakage.

Step 2 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same.
- **HINTS:**
  - `from sklearn.feature_selection import VarianceThreshold`.
  - `VarianceThreshold(threshold=0.01)` then `.fit_transform(X_train[numeric_cols])`.

Step 3 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where |corr| > 0.9.
- **HINTS:**
  - Compute correlation matrix.
  - Get the upper triangle with `np.triu(...)`.
  - For each column, if any of its upper-triangle correlations is > 0.9, mark it for dropping.

Step 4 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about `is_late`.
- **HINTS:**
  - `from sklearn.feature_selection import mutual_info_classif`.
  - `mi = mutual_info_classif(X_train_numeric, y_train)`.
  - Put in a Series, sort.
- **EXPECTED:** `distance_km`, `estimate_days`, `seller_avg_late_rate` should be at the top.

Step 5 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF, look at `feature_importances_`.
- **HINTS:**
  - `from sklearn.ensemble import RandomForestClassifier`.
  - `RandomForestClassifier(n_estimators=50, max_depth=8, n_jobs=-1)`.
  - `.fit(X_train, y_train)`.
  - Look at `.feature_importances_`.

Step 6 — Pick the final 10–15 columns

- **WHAT:** Combine the rankings. Pick columns that:
  - Are high in mutual info, AND
  - Are high in RF importance, AND
  - Were not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

Step 7 — Save

- **HINTS:** Keep only the selected columns + `is_late`. Save as `orders_step4.parquet` to Drive.

Phase C — Make ONE chart for Beatriz

Beatriz's chart — "These are the 10 most important columns"

A horizontal bar chart of your top 10 features and their importance score.

- **HINTS:**
  - Use the RF importance.
  - `sort_values()` then `.head(10)` then `.plot.barh()`.
- **Title:** "Top 10 predictors of late delivery."
- **Y-axis:** column names.
- **X-axis:** Random Forest importance.
- **Takeaway:** "Distance and the estimated delivery time predict 70% of lateness. Everything else is small."

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on the FULL data	Leakage.
Drop columns without writing why	Module 4 students will not understand.
Keep too many columns (40+)	Slow training. Overfitting risk.
Drop a high-mutual-info column	Big mistake. Check why it is high before dropping.

Self-check before Class 5

- ☐ Train/test split done FIRST.
- ☐ 10–15 columns remain + `is_late`.
- ☐ You wrote down WHY for each kept column.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Beatriz.
- ☐ `orders_step4.parquet` saved to Drive.

Class 5 — Pipelines

> **Scenario reminder:** Beatriz says: "Your cleaning code is in 4 different notebooks. When a new order arrives tomorrow, you cannot copy 4 notebooks to the server. We need ONE object that does everything."

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code.

Inputs

- `orders_step4.parquet` (selected columns)

Outputs

- `olist_pipeline.joblib` saved in Drive
- 1 confusion-matrix chart + 1 ROC curve chart
- A pipeline that takes RAW input and produces predictions

Phase A — Explore the data first

Exploratory chart 1 — Confusion matrix (after training)

- After Step 6, build a confusion matrix.
- **HINTS:**
  - `from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay`.
  - `cm = confusion_matrix(y_test, y_pred)`.
  - `ConfusionMatrixDisplay(cm, display_labels=['On time', 'Late']).plot()`.
- **What you learn:** How many late orders did we CATCH? How many did we MISS?

Exploratory chart 2 — ROC curve

- **HINTS:**
  - `from sklearn.metrics import RocCurveDisplay`.
  - `RocCurveDisplay.from_estimator(pipeline, X_test, y_test)`.
- **What you learn:** The trade-off between recall and false alarms. The AUC number.

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
  - numeric: `distance_km`, `estimate_days`, `total_price`, `total_freight`, `num_items`, `purchase_hour`, `log_freight`.
  - categorical: `payment_type`, `customer_state`.

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
  - `from sklearn.pipeline import Pipeline`.
  - `from sklearn.impute import SimpleImputer`.

  - `from sklearn.preprocessing import StandardScaler`.
  - Skeleton:
    ```
    numeric_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='___')),
        ('scaler',  StandardScaler()),
    ])
    ```
  - Fill in the blank: best strategy for numeric? (`'median'`)

Step 3 — Build the categorical mini-pipeline

- **WHAT:** 2 steps: imputer + one-hot encoder.
- **HINTS:**
  - `from sklearn.preprocessing import OneHotEncoder`.
  - Skeleton:
    ```
    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='___')),
        ('onehot',  OneHotEncoder(handle_unknown='___', sparse_output=___)),
    ])
    ```
  - Fill in blanks. (Strategy `'most_frequent'`, handle_unknown `'ignore'`, sparse_output `False`.)
- **WHY `handle_unknown='ignore'`?** In production, a new customer state may appear. We do not want the code to crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
  - `from sklearn.compose import ColumnTransformer`.
  - It takes a list of tuples: `(name, transformer, list_of_columns)`.

Step 5 — Add the model on top

- **HINTS:**
  - `from sklearn.linear_model import LogisticRegression`.
  - `LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)`.
  - Put it in a Pipeline with the preprocessor.
- **WHY `class_weight='balanced'`?** Only 7% of orders are late. Without this, the model just predicts "not late" for everyone and gets 93% accuracy (but useless).

Step 6 — Train and evaluate

- **HINTS:**
  - `full_pipeline.fit(X_train, y_train)`.
  - `y_pred = full_pipeline.predict(X_test)`.
  - `from sklearn.metrics import classification_report`.
- **EXPECTED:** F1 on the late class around 0.30–0.45. (Module 4 improves this.)

Step 7 — Save the trained pipeline

- **HINTS:**
  - `import joblib`.
  - `joblib.dump(full_pipeline, '/content/drive/MyDrive/olist_lab/olist_pipeline.joblib')`.

Phase C — Make ONE chart for Beatriz

Beatriz's chart — "How many late orders did we catch?"

A simple table or annotated bar:

	Predicted on time	Predicted late
Actually on time	true negatives	false alarms
Actually late	missed late	caught late

- **HINTS:** Just print the confusion matrix with labels. Or use `seaborn.heatmap()` on it.
- **Title:** "Of 1,000 late orders, our baseline model catches ~400."
- **Takeaway for Beatriz:** "Not great, but better than zero (which is what we have now). Module 4 will improve this with a better model."

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Forget `sparse_output=False` in OneHotEncoder	Some downstream code breaks.
Use `class_weight='balanced'` AND SMOTE	Over-correction. Pick one.
Forget `random_state`	Results change every run. Hard to debug.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
- ☐ Preprocessor has numeric + categorical transformers.
- ☐ `handle_unknown='ignore'` set.
- ☐ `class_weight='balanced'` set.
- ☐ Confusion matrix + ROC curve charts.
- ☐ `olist_pipeline.joblib` saved to Drive.

Class 6 — End-to-End Lab (THE BIG ONE)

> **Scenario reminder:** Beatriz is in the meeting room. She wants the FINAL clean dataset on her desk in 90 minutes. This is the lab.

Your goal

Take 9 raw CSV files. Produce ONE final `.parquet` file with the exact 21-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Inputs

- The 9 raw CSV files in your Drive folder

Outputs

- `olist_clean.parquet` (~95,000 rows × 21 columns) in Drive
- `findings.md` (~1 page)
- Your full Colab notebook

Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1–5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. `is_late` distribution (the 7% problem) 2. Distance vs late rate (your most important chart) 3. Top 10 most important features

These 3 charts tell Beatriz the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

Your `olist_clean.parquet` MUST have these 21 columns, with these names and types:

#	Column	Type	Source
1	`order_id`	string	orders
2	`customer_unique_id`	string	customers (joined)
3	`customer_state`	string	customers
4	`seller_state`	string	sellers (joined)
5	`purchase_year`	int	engineered
6	`purchase_month`	int	engineered
7	`purchase_dayofweek`	int	engineered
8	`purchase_hour`	int	engineered
9	`is_weekend`	int (0/1)	engineered
10	`num_items`	int	aggregated from items
11	`total_price`	float	aggregated
12	`total_freight`	float	aggregated
13	`log_freight`	float	engineered
14	`payment_type`	string	payments
15	`payment_installments`	int	payments
16	`distance_km`	float	Haversine
17	`delivery_days`	float	engineered (for analysis only — not a feature)
18	`estimate_days`	float	engineered
19	`is_late`	int (0/1)	TARGET (engineered)
20	`review_score`	int 1–5	reviews
21	`review_comment_message`	string	reviews (kept for Module 7!)

Lab phases (timed)

Phase	Time	What you do
1. Load	10 min	Load all 9 CSVs. Confirm row counts.
2. Clean	15 min	Convert dates, filter to delivered, drop missing delivery dates.

Phase	Time	What you do
3. Aggregate	15 min	Sum items per order. Pick primary payment.
4. Geo + Haversine	20 min	Build zip->latlon map. Join. Compute distance.
5. Date features + target	10 min	Engineer purchase_year/month/dow/hour/is_weekend, delivery_days, estimate_days, **is_late**.
6. Join reviews + save	10 min	Join reviews. Validate schema. Save `.parquet`.
7. Findings	10 min	Write `findings.md`.

Total: 90 minutes.

Phase C — Findings report for Beatriz (10 minutes)

Write `findings.md` with these sections:

Final numbers

- Final row count: _____
- `is_late` rate: _____% (should be 5–10%)

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____

One chart that summarizes everything

Embed your most important chart (the distance vs late rate one).

Grading rubric (100 points)

Item	Points
All 21 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline is reproducible (re-run from 9 CSVs in ONE command)	15
Haversine distance correct (within 1 km of the geopy library)	10
`is_late` rate is between 5–10%	10
At least 3 exploratory + 1 explanatory chart per class	**15**
`findings.md` has insights, not just numbers	10
Code is clean (comments, sensible variable names)	5

Item	Points

Submit

Upload to `module-3/class_6/submissions/<YourTeamName>/`: - `olist_clean.parquet` - Your Colab notebook (`File > Download > Download .ipynb`) - `findings.md`

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate `.parquet` files to Drive** (orders_step1, orders_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.

Good luck.